~ Complete Step by Step ~

MongoDB

NoSQL Database • Queries • Aggregation • Diagrams

Index

1 Introduction to MongoDB
13 Transactions & ACID
2 Installation & Setup
14 Replication (Replica Sets)
3 Data Model & BSON Types
15 Sharding
4 CRUD – Create
16 Security
5 CRUD – Read (find)
17 Backup & Restore
6 Query Operators
18 Performance & Monitoring
7 Update & Delete
19 Atlas, Compass & Tools
8 Arrays & Embedded Documents
20 Node.js Driver & Mongoose
9 Indexes
21 Advanced Features
10 Aggregation Framework
22 Practice Project (shell)
11 Data Modeling
23 Interview Q&A
12 Schema Validation
24 Cheat Sheet

How to use

- Helps to know **JSON** and basic **JavaScript** – the MongoDB shell (**mongosh**) uses JavaScript syntax.
- Type every example in **mongosh** or **Compass** on a free **Atlas** cluster. Redraw the **diagrams** in your notebook.
- Examples use a small ***shop*** database: ***users***, ***products***, ***orders***. Written for MongoDB **7 / 8**.

Notes by asadokz18 ✿

1. Introduction to MongoDB

Definition

MongoDB is an open-source, **document-oriented NoSQL database**. It stores data as flexible **JSON-like documents (BSON)** instead of rows and columns. Name comes from "humongous" (huge data). Created by **10gen (now MongoDB Inc.)** in 2009; written in C++. It powers apps at Google, Adobe, eBay, Forbes and many start-ups.

SQL (relational)

users

id	name	age	city
1	Asad	21	Lahore
2	Ali	25	Karachi

orders (separate table + JOIN)

id	user_id	item	price

fixed columns · schema first

MongoDB (document)

```
{
_id: ObjectId("65f..."),
name: "Asad", age: 21,
address: { city: "Lahore" },
skills: ["JS", "Node"],
orders: [ {item:"Book", price:10} ]
}
```

flexible JSON-like · nested data in ONE document

What is NoSQL?

"Not only SQL" – databases that don't use the fixed table model. Types: Document (MongoDB, CouchDB) · Key-value (Redis, DynamoDB) · Column-family (Cassandra) · Graph (Neo4j) · Search / Time-series.

Key features

- → **Flexible schema** – documents in one collection can differ
- → **Rich query language** – filters, regex, geo, text
- → **Indexes** on any field for speed
- → **Aggregation pipeline** for analytics
- → **Horizontal scaling** – sharding
- → **High availability** – replica sets, auto failover
- → **Transactions** (multi-document ACID since 4.0)
- → **Cloud** (Atlas), drivers for all languages

	SQL (MySQL / PostgreSQL)	MongoDB
Data unit	Table → rows	Collection → documents
Schema	Fixed, defined first	Dynamic (optional validation)
Relations	JOINs, foreign keys	Embedding or references; **$lookup**
Query language	SQL	MQL (JSON-style) + aggregation
Scaling	Mostly vertical	Horizontal (sharding) built in
Best for	Complex relations, strict consistency	Fast-changing, hierarchical, big-volume data

Terminology map

SQL	Database	Table	Row	Column	Index	JOIN	GROUP BY
MongoDB	Database	Collection	Document	Field	Index	$lookup / embedding	$group

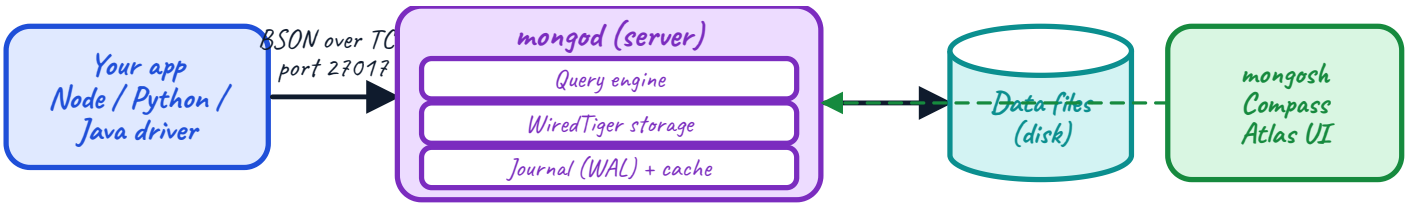
Good for

Content management, catalogs, user profiles, IoT / time-series, real-time analytics, mobile & web back-ends, big data.

Think twice when

You need many complex multi-table JOINs, strict relational integrity everywhere, or heavy financial reporting – SQL may fit better (MongoDB can still do transactions).

2. Installation & Setup



Option 1 – MongoDB Atlas (cloud, free, easiest)

1. Sign up at **mongodb.com/atlas**. 2. Create a free **M0 cluster** (choose cloud + region). 3. **Database Access** → add a user (username + password). 4. **Network Access** → allow your IP (or 0.0.0.0/0 for learning only). 5. **Connect** → copy the connection string:

connection string (SRV)

```
mongodb+srv://asad:<password>@cluster0.abcde.mongodb.net/shop?retryWrites=true&w=majority
```

Option 2 – Install locally (Community Server)

OS	How
Windows	Download **MSI** from mongodb.com/try/download/community → install as a **Service** + **MongoDB Compass** (GUI)
macOS	***brew tap mongodb/brew && brew install mongodb-community && brew services start mongodb-community***
Ubuntu	Add the MongoDB apt repo → ***sudo apt install -y mongodb-org*** → ***sudo systemctl start mongod***
Docker	***docker run -d --name mongo -p 27017:27017 -v mongodata:/data/db mongo:7***

Connect with the shell (mongosh)

```
mongosh                                          # connects to mongodb://127.0.0.1:27017
mongosh "mongodb+srv://cluster0.abcde.mongodb.net/shop" --username asad      # Atlas
mongod --dbpath /data/db --port 27017            # start server manually  (default port 27017, data in /data/db)
```

First commands

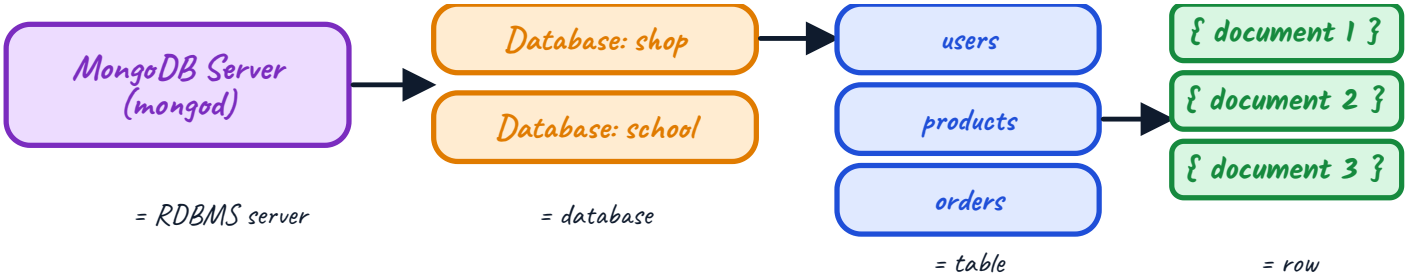
```
show dbs                    // list databases          db          // current database
use shop                    // switch (created when you insert first data)
show collections            // list collections        db.stats()  // size info
db.users.insertOne({ name: "Asad", age: 21 })   // creates "users" collection automatically
db.users.find()             // → [ { _id: ObjectId("..."), name: 'Asad', age: 21 } ]
db.dropDatabase()           // delete current database ⚠      db.help() · exit / quit()
```

Tool	Purpose
mongosh	Modern JavaScript shell (replaces old **mongo**)
MongoDB Compass	Free GUI: browse data, build queries / aggregations visually, indexes, explain plans
mongodump / mongorestore	Backup / restore (BSON) · ***mongoexport / mongoimport*** for JSON / CSV
VS Code extension	MongoDB for VS Code – playgrounds (.mongodb files)

Good to know

Default: port **27017**, no authentication when started plainly – **enable auth** before exposing it (Chapter 16). Databases and collections are created **lazily**. Reserved DBs: ***admin***, ***local***, ***config***.

3. Data Model – Database, Collection, Document & BSON



→ **Database** – container for collections (**shop**). **Collection** – group of documents, like a table but **schema-less**.
Document – one record: ordered **field: value** pairs, max size **16 MB**.

A document – nested objects & arrays

```
{
  _id: ObjectId("65f1c2a8e4b0a1b2c3d4e5f6"),          // primary key – auto-added, unique
  name: "Asad Khan",  age: 21,  isActive: true,  balance: Decimal128("120.50"),
  email: "asad@example.com",
  address: { city: "Lahore", country: "PK", geo: { type: "Point", coordinates: [74.35, 31.52] } },   // embedded document
  skills: ["JavaScript", "Node.js", "MongoDB"],                                                       // array
  orders: [ { item: "Book", qty: 2, price: 15.5 }, { item: "Pen", qty: 10, price: 1 } ],              // array of documents
  createdAt: new Date(),  avatar: null
}
```

BSON – Binary JSON

MongoDB stores documents as **BSON**: binary, fast to scan, and with **more types than JSON** (Date, ObjectId, Int32/64, Decimal128, Binary).

Type	Example	Note
String	"hello"	UTF-8
Int32 / Int64 / Double	NumberInt(5) · NumberLong(9e12) · 3.14	JS numbers are Double by default
Decimal128	NumberDecimal("19.99")	Exact – use for **money**
Boolean / Null	true · null	
Date	new Date() · ISODate("2026-10-20")	UTC milliseconds
ObjectId	ObjectId("65f1...")	12-byte unique id
Array / Object	[1,2] · {"a":1}	Nesting allowed
BinData / Regex / Timestamp	BinData(...) · /^abc/i	Files bytes · patterns · internal

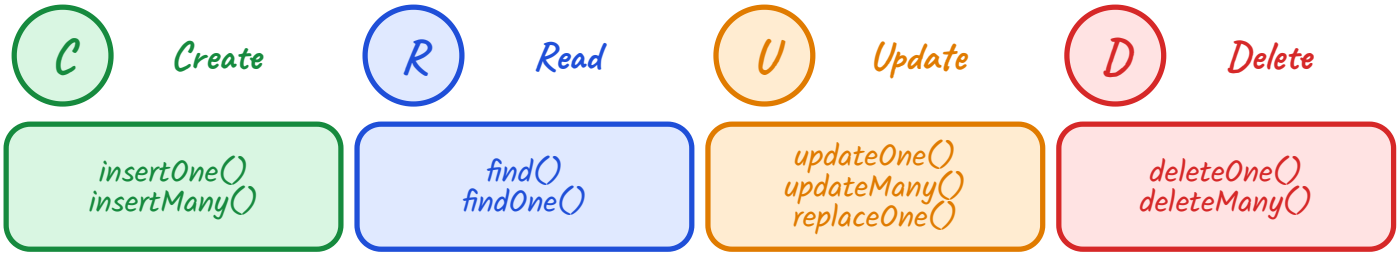
ObjectId – the default _id

4 bytes	5 bytes	3 bytes
Timestamp (seconds)	**Random** (machine/process)	**Counter** (incrementing)

```
const id = new ObjectId();      id.getTimestamp();      id.toString();      // creation time is inside the id!
db.users.find({ _id: ObjectId("65f1c2a8e4b0a1b2c3d4e5f6") })                // must wrap strings in ObjectId()
db.users.insertOne({ _id: "custom-1", name: "X" })                          // _id can be any unique value (not an array)
```

Rules: every document has a unique **_id** · max **16 MB** · avoid **$** or **.** in field names · collections plural lowercase, fields camelCase.

4. CRUD Basics & Create (Insert)



insertOne() – add a single document

```
use shop
db.users.insertOne({ name: "Asad", age: 21, city: "Lahore", skills: ["JS", "Node"], createdAt: new Date() })
// → { acknowledged: true, insertedId: ObjectId("6Sf1...") }
```

insertMany() – add many (array)

```
db.users.insertMany([
  { name: "Ali",   age: 25, city: "Karachi", skills: ["Python"] },
  { name: "Sara",  age: 19, city: "Lahore",  skills: ["React", "CSS"] },
  { name: "Bilal", age: 30, city: "Islamabad" },                    // different fields are fine – flexible schema
  { name: "Hina",  age: 28, city: "Karachi", skills: ["Java", "SQL"] }
])
// → { acknowledged: true, insertedIds: { '0': ObjectId(..), '1': ObjectId(..), ... } }
```

Option / behaviour	Meaning
insertMany(docs, {"{ ordered: false }"})	**ordered:true (default)** stops at the first error; **false** continues with the rest
Duplicate _id / unique key	Error **E11000 duplicate key error**
Auto-create	Collection & database are created on first insert
writeConcern	**{"{ w: \"majority\" }"}** – wait until a majority of nodes confirm

Import data from a file / bulk operations

```
// terminal:  mongoimport --db shop --collection products --file products.json --jsonArray      (CSV: --type csv --headerline)
db.products.bulkWrite([                                          // many DIFFERENT operations in one round-trip
  { insertOne: { document: { name: "Pen", price: 1, stock: 100 } } },
  { updateOne: { filter: { name: "Book" }, update: { $inc: { stock: -1 } } } },
  { deleteOne: { filter: { name: "Old" } } }
], { ordered: true })
```

Sample data used in these notes (copy & paste)

```
db.products.insertMany([
  { name: "Laptop",  price: 900, category: "electronics", stock: 15, tags: ["pc", "work"], rating: 4.5 },
  { name: "Phone",   price: 600, category: "electronics", stock: 30, tags: ["mobile"],       rating: 4.2 },
  { name: "Desk",    price: 150, category: "furniture",   stock: 8,  tags: ["office", "work"], rating: 4.0 },
  { name: "Chair",   price: 85,  category: "furniture",   stock: 0,  tags: ["office"],       rating: 3.8 },
  { name: "Notebook",price: 3,   category: "stationery",  stock: 500, tags: ["school"],      rating: 4.7 }
])
```

Tips

Use **insertMany** for bulk loads (far faster than many insertOne). Validate data in your app or with **schema validation** (Chapter 12). Keep documents under a few hundred KB for speed. Dates → ***new Date()***, not strings.

5. Read – find(), Projection, Sort, Limit



find() returns a CURSOR (lazy) – batches of 101 docs; .toArray() / for-await loads them

```
db.products.find()                                  // ALL documents (cursor, shows first 20 in mongosh → type "it" for more)
db.products.find({ category: "furniture" })         // filter: equality
db.products.find({ category: "electronics", price: { $lt: 700 } })      // several conditions = AND
db.products.findOne({ name: "Desk" })               // first match as ONE document (or null)
db.products.find({ "address.city": "Lahore" })      // dot notation for embedded fields (quotes required)
db.products.countDocuments({ stock: { $gt: 0 } })   // exact count  ·  estimatedDocumentCount() → fast, whole collection
db.products.distinct("category")                    // unique values → ["electronics","furniture","stationery"]
```

Projection – choose which fields to return

```
db.products.find({}, { name: 1, price: 1 })         // include name & price (+ _id by default)
db.products.find({}, { name: 1, price: 1, _id: 0 }) // hide _id
db.products.find({}, { tags: 0, rating: 0 })        // exclude fields  (can't mix 1 and 0, except _id)
db.products.find({}, { tags: { $slice: 1 } })       // first element of an array only
```

Sort, skip, limit – and pagination

```
db.products.find().sort({ price: 1 })               // 1 = ascending, -1 = descending
db.products.find().sort({ category: 1, price: -1 }) // sort by several fields
db.products.find().sort({ price: -1 }).limit(3)     // top 3 most expensive
db.products.find().skip(10).limit(10)               // page 2 (page size 10):  skip = (page-1) * size
db.products.find({ price: { $gt: 100 } }).sort({ price: 1 }).limit(2).skip(1).toArray()   // order of chaining doesn't matter
```

Method	Purpose
find(filter, projection)	Returns a **cursor** to matching documents
findOne()	First matching document (no cursor)
.sort() .skip() .limit()	Order • skip N • max N documents (applied **sort → skip → limit**)
.count() / countDocuments()	Number of matches (**.count()** is deprecated)
.toArray() / .forEach() / .map()	Load cursor results • iterate: **cursor.forEach(d => print(d.name))**
.explain("executionStats")	See how the query ran (index used? docs scanned?)
.pretty() / .hasNext() / .next()	Old formatting (auto now) • manual cursor stepping

skip() is slow for big offsets

MongoDB still walks past skipped docs. For deep pages use **range pagination**: **find({"{_id: {$gt: lastId}}"}).sort({"{_id:1}"}).limit(10)**.

Remember

Filters are **case-sensitive** and **type-sensitive**: **"21" ≠ 21**. Missing fields don't match. Cursors time out after ~10 min idle.

6. Query Operators

Operators start with $ and describe *how* to match. Syntax: `{"{ field: { $operator: value } }"}`

Comparison	Logical	Element	Array	Evaluation
$eq $ne $gt $gte $lt $lte $in $nin	$and $or $not $nor	$exists $type	$all $size $elemMatch	$regex $expr $mod $text $where

Comparison

```
db.products.find({ price: { $gt: 100 } })                    // $gt >   $gte >=   $lt <   $lte <=
db.products.find({ price: { $gte: 100, $lte: 700 } })        // range: 100 ≤ price ≤ 700   (same field, many operators)
db.products.find({ category: { $ne: "furniture" } })         // not equal (also matches docs without the field)
db.products.find({ category: { $in: ["electronics", "stationery"] } })    // value in list   |   $nin = not in list
```

Logical

```
db.products.find({ $or: [ { category: "furniture" }, { price: { $lt: 10 } } ] })        // either condition
db.products.find({ $and: [ { price: { $gt: 50 } }, { stock: { $gt: 0 } } ] })           // (comma list is already AND)
db.products.find({ price: { $not: { $gt: 100 } } })                                     // negate an operator expression
db.products.find({ $nor: [ { category: "furniture" }, { stock: 0 } ] })                 // none of the conditions
```

Element, evaluation & array

```
db.users.find({ skills: { $exists: true } })          // field present   (false = missing)      $type: "string" | "int" | "array" | "date"
db.users.find({ age: { $type: "number" } })
db.users.find({ name: { $regex: "^A", $options: "i" } })  // regex   (or  { name: /^a/i } )   - anchored ^ patterns can use an index
db.products.find({ $expr: { $gt: ["$price", { $multiply: ["$cost", 1.5] }] } })          // compare two fields of the SAME document
db.products.find({ stock: { $mod: [5, 0] } })          // stock divisible by 5
db.products.find({ tags: "work" })                     // array contains value      |   { tags: { $all: ["office","work"] } } contains ALL
db.products.find({ $text: { $search: "laptop work" } })  // text search (needs a text index, Chapter 9)
```

Goal	Query
Cheap OR out of stock	`{"{ $or: [{price:{$lt:10}}, {stock:0}] }"}`
Name starts with "Ch"	`{"{ name: /^Ch/ }"}`
Between two dates	`{"{ createdAt: { $gte: ISODate('2026-01-01'), $lt: ISODate('2026-02-01') } }"}`
Field is null or missing	`{"{ phone: null }"}` vs only missing: `{"{ phone: {$exists:false} }"}`
Multiple conditions same field	`{"{ $or:[{city:'A'},{city:'B'}] }"}` or `{"{ city:{$in:['A','B']} }"}`

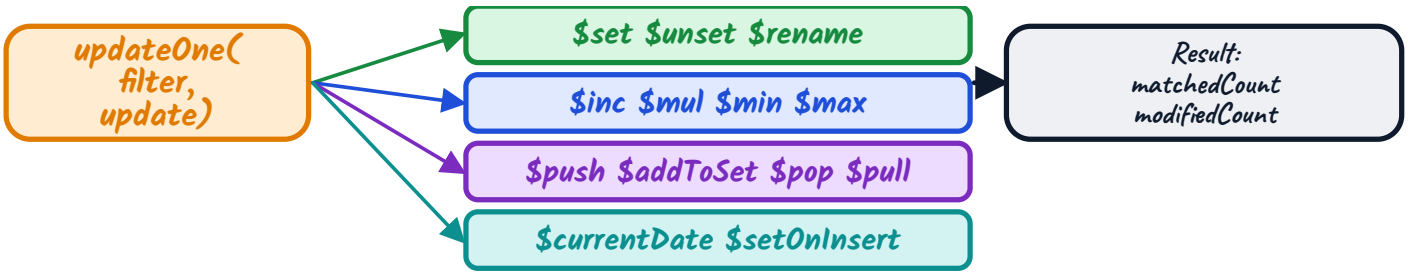
Common mistake

Duplicate keys overwrite: `{"{price:{$gt:5}, price:{$lt:9}}"}` keeps only the last. Put both operators in **one** object: `{"{price: {$gt:5,$lt:9}}"}`.

Geo & bitwise

$near, **$geoWithin**, **$geoIntersects** (2dsphere index) · **$bitsAllSet** … · **$where** (JS, slow - avoid).

7. Update & Delete



Always include an update OPERATOR ($set ...) – without it the whole document is replaced

updateOne / updateMany – use operators!

```
db.users.updateOne({ name: "Asad" }, { $set: { age: 22, city: "Islamabad" } })          // change/add fields, keep the rest
db.users.updateMany({ city: "Karachi" }, { $set: { country: "PK" } })                    // every match
db.users.updateOne({ name: "Ali" }, { $inc: { age: 1 } })                                // increment (negative = decrement)
db.products.updateMany({}, { $mul: { price: 1.1 } })                                     // multiply: +10 %
db.users.updateOne({ name: "Sara" }, { $unset: { city: "" } })                           // remove a field
db.users.updateMany({}, { $rename: { city: "town" } })                                   // rename field
db.products.updateOne({ name: "Desk" }, { $min: { price: 100 }, $max: { rating: 4.9 }, $currentDate: { updatedAt: true } })
// → { acknowledged: true, matchedCount: 1, modifiedCount: 1, upsertedCount: 0 }
```

Array operator	Effect
$push: {"{ skills: \"Go\" }"}	Append to array (duplicates allowed). **$push: {"{ s: { $each: [\"a\",\"b\"], $slice: -5 } }"}**
$addToSet: {"{ skills: \"Go\" }"}	Add only if **not already present**
$pull: {"{ skills: \"SQL\" }"}	Remove all matching elements · **$pullAll** list
$pop: {"{ skills: 1 }"}	Remove last (1) or first (-1) element
"skills.$": "Rust" with filter	**Positional $** – update the first array element that matched the filter

upsert, replace & find-and-modify

```
db.users.updateOne({ email: "new@x.com" }, { $set: { name: "New" }, $setOnInsert: { createdAt: new Date() } }, { upsert: true })   // update, or INSERT if no match
db.users.replaceOne({ name: "Hina" }, { name: "Hina", age: 29, role: "admin" })          // replace WHOLE doc (keeps _id)
db.counters.findOneAndUpdate({ _id: "orderNo" }, { $inc: { seq: 1 } }, { returnDocument: "after", upsert: true })
// atomic: get new value back
db.users.updateOne({ name: "Asad", "orders.item": "Book" }, { $set: { "orders.$.qty": 5 } })
// positional
db.users.updateOne({ name: "Asad" }, { $set: { "scores.$[s].pts": 100 } }, { arrayFilters: [ { "s.sub": "math" } ] })   // all matching elements
```

Delete

```
db.users.deleteOne({ name: "Bilal" })          // first match → { acknowledged: true, deletedCount: 1 }
db.users.deleteMany({ age: { $lt: 18 } })      // all matches
db.users.deleteMany({})                        // ALL documents (collection & indexes remain)
db.users.drop()                                // delete the whole collection ⚠        db.dropDatabase()
⚠
db.users.findOneAndDelete({ name: "Ali" })     // delete and return the deleted document
```

Careful: without **$set** etc. an update errors (v4.2+). Test the filter with ***find()*** before ***deleteMany***. A write to **one document** is always **atomic**.

8. Querying Arrays & Embedded Documents

```
{ name: "Asad",
skills: ["JS", "Node", "SQL"],
scores: [ {sub:"math", pts:90},
{sub:"cs", pts:75} ],
address: { city: "Lahore", zip: 54000 } }
```

```
skills: "JS"                                  contains value
skills: {$all:["JS","SQL"]}                   has ALL
skills: {$size: 3}                            length = 3
"skills.0": "JS"                              by index
scores: {$elemMatch:{sub:"cs",pts:{$gt:70}}}  same element matches BO
"address.city": "Lahore"                      dot notation
```

Embedded documents – dot notation

```
db.users.find({ "address.city": "Lahore" })                    // field inside a sub-document  (QUOTES needed)
db.users.find({ address: { city: "Lahore", country: "PK" } })  // exact match of the WHOLE sub-document (order matters!) – usually not wanted
db.users.find({ "address.geo.coordinates.0": { $gt: 70 } })    // deep path + array index
db.users.updateOne({ name: "Asad" }, { $set: { "address.zip": 54000 } })   // update one nested field only
```

Arrays – queries

```
db.users.find({ skills: "Node.js" })                  // array CONTAINS this value
db.users.find({ skills: ["JS", "Node"] })             // array EQUALS exactly (same order) – rare
db.users.find({ skills: { $all: ["JS", "Node"] } })   // contains both (any order)
db.users.find({ skills: { $in: ["Go", "Rust"] } })    // contains at least one of these
db.users.find({ skills: { $size: 3 } })               // exactly 3 elements  (no ranges → store a count field)
db.users.find({ "skills.0": "JS" })                   // first element is JS
db.users.find({ scores: { $gt: 80 } })                // ANY element > 80 (for array of numbers)
db.users.find({}, { skills: { $slice: 2 } })          // projection: first 2;  { $slice: -2 } last 2;  { $slice: [1,2] } skip 1 take 2
```

$elemMatch – conditions on the SAME array element

```
// scores: [ {sub:"math", pts:90}, {sub:"cs", pts:60} ]
db.users.find({ "scores.sub": "cs", "scores.pts": { $gt: 80 } })                  // ✘ matches! (cs from one element, 90 from another)
db.users.find({ scores: { $elemMatch: { sub: "cs", pts: { $gt: 80 } } } })        // ✔ one element must satisfy BOTH
db.users.find({ scores: { $elemMatch: { sub: "cs" } } }, { "scores.$": 1 })       // return only the first matching element
```

Task	How
Add / remove elements	**$push · $addToSet · $pull · $pop**
Update matched element	**"arr.$" · "arr.$[]" (all) · "arr.$[x]" + arrayFilters**
Flatten array in a query	**aggregate([{"{ $unwind: \"$skills\" }"}])**
Array of arrays	Possible, but avoid – hard to query

Embedded vs separate collection

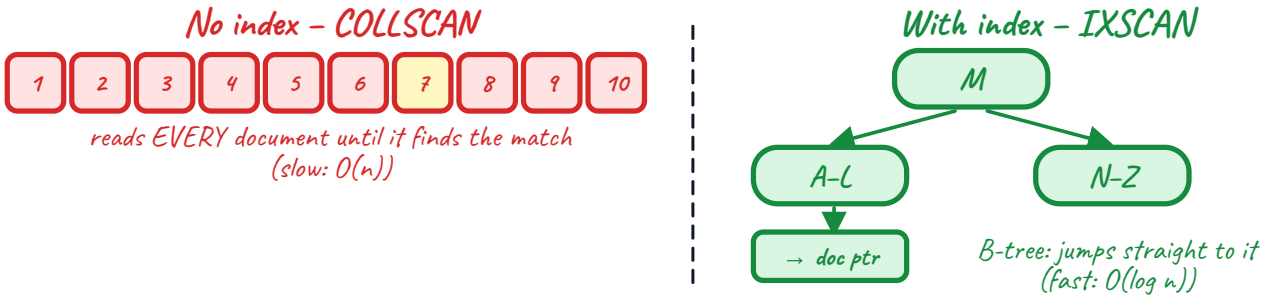
Embed things that are **read together** and are **bounded** (address, a few phone numbers). Don't embed unbounded lists (thousands of comments).

Gotchas

Dot-notation needs **quotes** · exact sub-document match is order-sensitive · **$size** can't use ranges · a multikey index is created automatically for array fields.

9. Indexes

An **index** is a small sorted data structure (**B-tree**) that lets MongoDB find documents **without scanning the whole collection**. Like the index of a book. Every collection has an index on **_id**.




```
db.users.createIndex({ email: 1 })                                  // single field   (1 asc, -1 desc)
db.users.createIndex({ email: 1 }, { unique: true })                // no duplicates allowed (E11000 on violation)
db.products.createIndex({ category: 1, price: -1 })                 // COMPOUND index – order of fields matters!
db.products.createIndex({ name: "text", description: "text" })      // TEXT index → $text search
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })     // TTL: auto-delete docs after 1 hour
db.users.createIndex({ phone: 1 }, { sparse: true })                // only docs that have the field
db.users.createIndex({ status: 1 }, { partialFilterExpression: { status: "active" } })     // index only some docs → smaller
db.places.createIndex({ location: "2dsphere" })                     // GEO queries ($near, $geoWithin)
db.users.getIndexes();    db.users.dropIndex("email_1");    db.users.dropIndexes()
```

Index type	Use
Single field	Queries / sorts on one field
Compound	Several fields – supports **prefixes**: {"{a,b,c}"} serves a · a,b · a,b,c (not b alone)
Multikey	Auto for array fields (one entry per element)
Text	Word search with stemming & score (1 text index per collection)
2dsphere / 2d	Geospatial (maps, "near me")
Hashed	Equality only; used for **hash sharding**
TTL / Unique / Partial / Wildcard	Expire data · enforce uniqueness · index subset · index unknown fields

explain() – is my query using the index?

```
db.products.find({ category: "furniture" }).sort({ price: -1 }).explain("executionStats")
// winningPlan.stage:  COLLSCAN ✘ (full scan)   |   IXSCAN ✔ (index)   →   FETCH
// compare  totalDocsExamined  vs  nReturned  (should be close!)   executionTimeMillis
db.products.find({ name: "Pen" }).hint({ name: 1 })        // force an index (testing)
```

ESR rule (compound indexes)

Order fields: Equality → Sort → Range. Query **{"{status:'A', price:{$gt:5}}"}** sorted by date → index **{"{status:1, date:1, price:1}"}**.

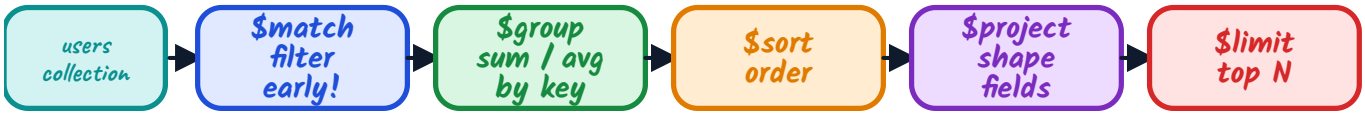
Covered query: all needed fields are in the index → no document fetch.

Indexes cost

Each index uses **RAM + disk** and slows **writes**. Index what you query & sort by; drop unused ones (**$indexStats**). Anchored regex **/^abc/** uses an index, **/abc/** doesn't well.

10. Aggregation Framework

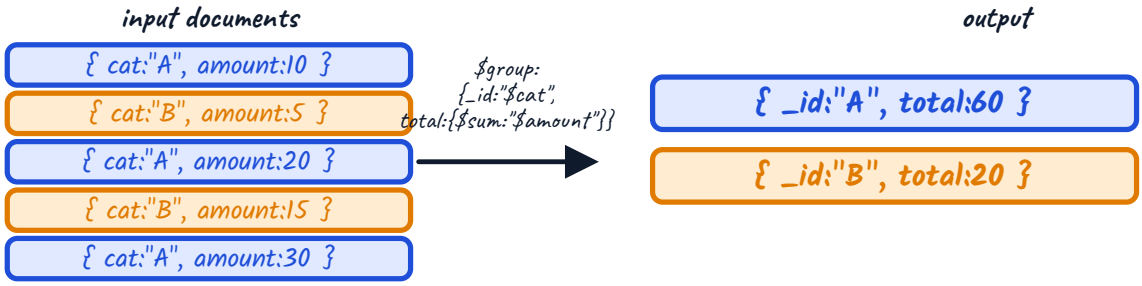
An **aggregation pipeline** processes documents through **stages** – the MongoDB way to do **GROUP BY, JOIN, reports and analytics**. Syntax: ***db.col.aggregate([stage1, stage2, ...])***



documents flow through STAGES – each stage transforms them and passes the result on

Example: total sales per category, best first

```
db.products.aggregate([
  { $match: { stock: { $gt: 0 } } },                                   // 1. filter (same syntax as find) – put FIRST to use indexes
  { $group: { _id: "$category",                                        // 2. group key (note the $ before field names)
              totalStock: { $sum: "$stock" }, avgPrice: { $avg: "$price" },
              maxPrice: { $max: "$price" }, items: { $push: "$name" }, count: { $sum: 1 } } },
  { $sort: { totalStock: -1 } },                                       // 3. order
  { $project: { _id: 0, category: "$_id", totalStock: 1, avgPrice: { $round: ["$avgPrice", 2] }, count: 1 } },   // 4. reshape
  { $limit: 5 }                                                        // 5. top 5
])
```



Stage	Purpose
$match	Filter documents (like find) · **$limit** / **$skip** / **$sort** / **$sample**
$group	Group by **_id** and compute **$sum $avg $min $max $count $push $addToSet $first $last**
$project	Include / exclude / compute fields · **$addFields** / **$set** add computed fields · **$unset** · **$replaceRoot**
$unwind	Split an array into one document per element
$lookup	**Left outer JOIN** with another collection
$facet / $bucket	Several pipelines at once (dashboards) / histogram buckets
$out / $merge	Write results to a collection · **$count** · **$setWindowFields** (running totals)

10. Aggregation – $unwind, $lookup & more examples

$unwind – one document per array element

```
// { name:"Asad", skills:["JS","Node"] }   →   { name:"Asad", skills:"JS" }, { name:"Asad", skills:"Node" }
db.users.aggregate([
  { $unwind: "$skills" },
  { $group: { _id: "$skills", users: { $sum: 1 } } },          // how many users per skill?
  { $sort: { users: -1 } }
])
```

$lookup – JOIN orders with users

```
db.orders.aggregate([
  { $lookup: { from: "users",              // other collection
               localField: "userId",      // field in orders
               foreignField: "_id",       // field in users
               as: "user" } },            // output array
  { $unwind: "$user" },                   // array of 1 → object
  { $project: { _id: 0, item: 1, total: 1, "user.name": 1, "user.city": 1 } }
])
// $lookup with pipeline (conditions):  { $lookup: { from:"orders", let:{uid:"$_id"}, pipeline:[ {$match:{$expr:{$eq:
["$userId","$$uid"]}}}, {$limit:3} ], as:"recent" } }
```

Dates, conditions & computed fields

```
db.orders.aggregate([
  { $match: { createdAt: { $gte: ISODate("2026-01-01") } } },
  { $addFields: { year: { $year: "$createdAt" }, month: { $month: "$createdAt" },
                  size: { $cond: [ { $gte: ["$total", 100] }, "big", "small" ] } } },          // if / else
  { $group: { _id: { year: "$year", month: "$month" }, revenue: { $sum: "$total" }, orders: { $sum: 1 } } },
  { $sort: { "_id.year": 1, "_id.month": 1 } }
])
// other expressions:  $concat $toUpper $substr $dateToString $ifNull $switch $arrayElemAt $size $filter $map
$reduce
```

$facet & $bucket

```
db.products.aggregate([{ $facet: {
  byCategory: [{ $group: { _id: "$category", n: {
$sum: 1 } } }],
  priceRanges: [{ $bucket: { groupBy: "$price",
boundaries: [0, 50, 200, 1000], default: "other" } }],
  top3: [{ $sort: { rating: -1 } }, { $limit: 3 }]
}}])
```

Tip
$match & $sort first → can use indexes
$project early to reduce document size
Stage memory limit 100 MB → **{"{allowDiskUse:true}"}**
.explain() works on aggregate too
$merge saves report into a collection

SQL → MongoDB

WHERE = $match · **GROUP BY** = $group · **HAVING** = $match after $group · **SELECT** = $project · **ORDER BY** = $sort · **LIMIT** = $limit · **JOIN** = $lookup · **UNION** = $unionWith.

11. Data Modeling – Embed or Reference?

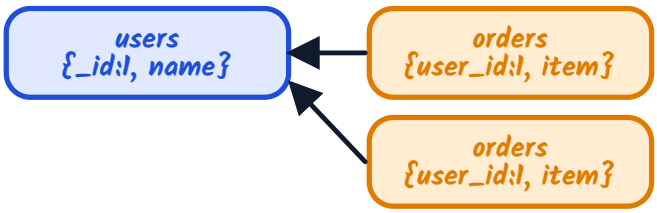
In MongoDB you design the schema around **how the application reads and writes data** ("data that is accessed together should be stored together").

EMBEDDING (denormalized)

```
{ _id: 1, name: "Asad",
address: { city, zip },
orders: [ {item,qty}, ... ] }
```

1 read gets everything · atomic update
but: duplicates data, 16 MB limit

REFERENCING (normalized)



no duplication, any size, many-to-many
but: 2 queries / $lookup

Relationship	Recommended model	Example
One-to-One	**Embed** the sub-document	user → profile / address
One-to-Few	**Embed** an array	person → 3 phone numbers
One-to-Many	**Reference** (child stores parent id) or embed if bounded	blog post → comments, user → orders
One-to-Squillions	**Parent reference** only (many docs)	host → millions of log entries
Many-to-Many	Array of ids on one/both sides	students ↔ courses

Embedding vs referencing – example

```
// EMBED (read together, bounded)
{ _id: 1, title: "Post",
  author: { name: "Asad", avatar: "a.png" },
  comments: [ { by: "Ali", text: "Nice" } ] }

// REFERENCE (separate collections, join with $lookup)
// posts:    { _id: 1, title: "Post", authorId: 7 }
// authors:  { _id: 7, name: "Asad", bio: "..." }
// comments: { postId: 1, by: "Ali", text: "Nice" }
```

	Embed when…	Reference when…
Reads	Data is read together (one query)	Parts are used independently
Size	Small & bounded	Large / unbounded growth (16 MB limit!)
Updates	Changes together (atomic)	Child changes often / shared by many parents

Useful design patterns

Pattern	Idea
Subset	Embed only the latest 10 reviews in the product; keep all in a separate collection
Extended reference	Copy a few fields (name) of the referenced doc to avoid joins
Computed	Pre-compute totals (***likesCount***, ***orderTotal***) with **$inc**
Bucket	Group time-series/IoT readings into one doc per hour/day
Schema versioning	Add ***schemaVersion*** field to migrate gradually
Outlier / Polymorphic	Special handling for huge docs / different shapes in one collection

Anti-patterns

Unbounded arrays that grow forever · huge documents · too many collections or indexes · storing everything in one blob · forcing a relational design (normalising every table) · no indexes on query fields.

12. Schema Validation

MongoDB is schema-flexible, but you can **enforce rules** on a collection with **$jsonSchema** (JSON Schema draft 4 subset) – required fields, types, ranges, enums.

Create a collection with a validator

```
db.createCollection("students", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["name", "email", "age"],
    properties: {
      name:  { bsonType: "string", minLength: 2, description: "must be a string, 2+ chars" },
      email: { bsonType: "string", pattern: "^\\S+@\\S+\\.\\S+$" },
      age:   { bsonType: "int", minimum: 5, maximum: 100 },
      grade: { enum: ["A", "B", "C", "D", "F"] },
      skills: { bsonType: "array", items: { bsonType: "string" }, uniqueItems: true },
      address: { bsonType: "object", required: ["city"], properties: { city: { bsonType: "string" } } }
    },
    additionalProperties: true                    // false = no unknown fields (remember to list _id)
  } },
  validationLevel: "strict",          // "strict" = all inserts &amp;amp; updates | "moderate" = skip existing invalid docs
  validationAction: "error"           // "error" = reject | "warn" = allow but log
})
```

Test it + modify later

```
db.students.insertOne({ name: "A", email: "bad", age: 3 })
// → MongoServerError: Document failed validation        (details: db.students.insertOne(...) shows errInfo)
db.students.insertOne({ name: "Asad", email: "asad@x.com", age: NumberInt(21), grade: "A" })      // ✔ ok
db.runCommand({ collMod: "students", validator: { $jsonSchema: { /* new rules */ } }, validationLevel: "moderate"
})   // change rules
db.getCollectionInfos({ name: "students" })[0].options.validator                                  // view rules
```

Keyword	Meaning
bsonType	"string" "int" "long" "double" "decimal" "bool" "date" "objectId" "array" "object"
required / properties	Mandatory fields / rules per field
minimum maximum minLength maxLength pattern	Number / string constraints
enum / items / uniqueItems	Allowed values / array element rules
oneOf anyOf allOf not	Combine schemas

Why validate?

Data quality at the **database level** – protects against bugs in any client (Node, Python, Compass). Use alongside app-level validation (Mongoose / Zod).

Mongoose

Mongoose schemas validate in the **application** (required, min, enum, custom). The DB validator adds a second safety net. Plain query-operator validators also work: **{"{ validator: { age: { $gte: 0 } } }"}**.

13. Transactions & ACID

A **transaction** groups several operations so they **all succeed or all fail**. Single-document writes are already atomic; since **MongoDB 4.0** multi-document ACID transactions work on replica sets (4.2: sharded clusters).



ACID: Atomic · Consistent · Isolated · Durable

ACID	Meaning (bank transfer example)
Atomicity	Debit A and credit B happen together or not at all
Consistency	Rules stay valid (balance never negative, unique keys)
Isolation	Other users don't see half-finished results (snapshot isolation)
Durability	Once committed it survives crashes (journal + write concern)

mongosh

```
const session = db.getMongo().startSession();
const accounts = session.getDatabase("bank").accounts;
session.startTransaction({ readConcern: { level: "snapshot" }, writeConcern: { w: "majority" } });
try {
  accounts.updateOne({ _id: "A" }, { $inc: { balance: -100 } });
  accounts.updateOne({ _id: "B" }, { $inc: { balance:  100 } });
  session.commitTransaction();                 // both saved
} catch (e) { session.abortTransaction(); throw e; }       // none saved
finally { session.endSession(); }
```

Node.js driver – withTransaction (auto-retry)

```
const session = client.startSession();
try {
  await session.withTransaction(async () => {
    await accounts.updateOne({ _id: "A" }, { $inc: { balance: -100 } }, { session });        // pass { session } to EVERY operation
    await accounts.updateOne({ _id: "B" }, { $inc: { balance:  100 } }, { session });
    await ledger.insertOne({ from: "A", to: "B", amount: 100, at: new Date() }, { session });
  });
} finally { await session.endSession(); }
```

Write concern & read concern

Setting	Meaning
w: 1	Acknowledged by the **primary** only (fast, risk of rollback if it fails)
w: "majority"	Confirmed by most nodes – **safe, default in recent versions**
j: true	Written to the on-disk **journal** before ack
readConcern "local" / "majority" / "snapshot"	Latest local data / only majority-committed data / consistent snapshot (transactions)
readPreference	**primary** (default) · **secondaryPreferred** · **nearest**

Costs

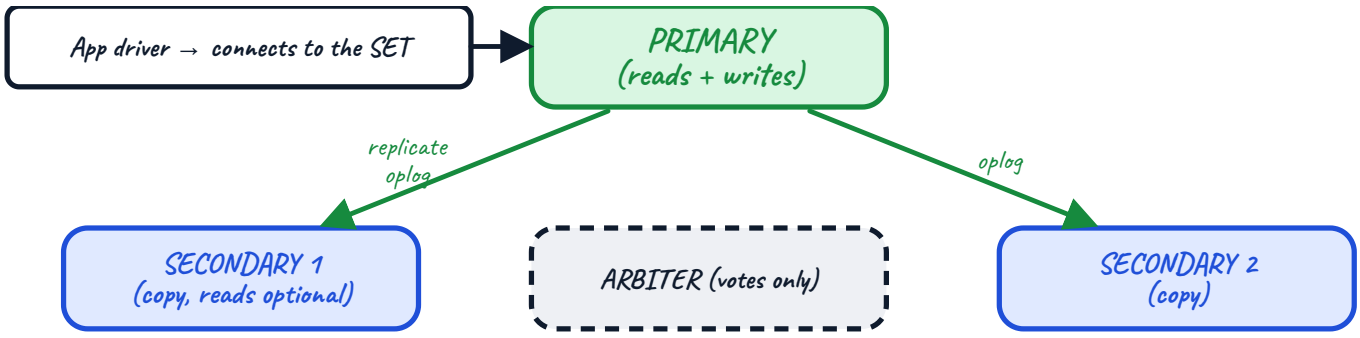
Transactions add overhead; default **60 s** limit; keep them **short**. Collections must already exist. Needs a **replica set** (even one node).

Prefer good modelling

Embed related data in one document → no transaction needed. Use transactions for money transfers, inventory + order, cross-collection consistency.

14. Replication – Replica Sets

A **replica set** is a group of ***mongod*** servers holding the **same data** for **high availability**, redundancy and read scaling. Minimum **3 members** (or 2 + arbiter) in production.



Primary down → secondaries ELECT a new primary (≈ 10 s) → no data loss, automatic failover

Member	Role
Primary	Only node accepting **writes** (and reads by default); records changes in the **oplog**
Secondary	Copies the primary's oplog asynchronously; can serve reads (***readPreference***); can become primary
Arbiter	No data, only votes in elections (avoid if possible)
Hidden / Delayed	Hidden: invisible to apps (reporting, backups) · Delayed: runs N hours behind to recover from human errors

Election & failover

1. Members send **heartbeats** every 2 s. 2. If the primary is unreachable for ~10 s, secondaries start an **election**. 3. A candidate needs a **majority of votes** (so use an **odd number** of voting members). 4. New primary is chosen; drivers reconnect automatically (***retryWrites=true***).

Set up a 3-node replica set (local test)

```
// terminal:  mongod --replSet rs0 --port 27017 --dbpath /data/r0   (also 27018, 27019 with other dbpaths)
rs.initiate({ _id: "rs0", members: [ { _id: 0, host: "localhost:27017" }, { _id: 1, host: "localhost:27018" }, { _id: 2, host: "localhost:27019" } ] })
rs.status()          rs.conf()          rs.isMaster()  /  db.hello()          rs.stepDown()       // force a new election
rs.add("localhost:27020")   rs.remove("localhost:27020")   rs.reconfig(cfg)
```

Connection string – list all members + replicaSet name

```
mongodb://host1:27017,host2:27017,host3:27017/shop?
replicaSet=rs0&readPreference=secondaryPreferred&w=majority&retryWrites=true
```

Why replicate?

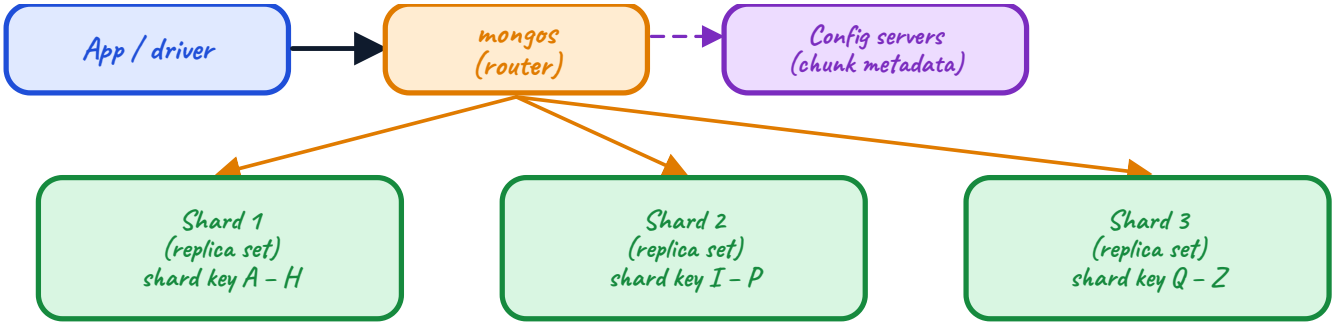
- **High availability** – automatic failover
- **Data safety** – several copies
- **Read scaling** & analytics on secondaries
- Required for **transactions** & change streams

Remember

- Replication is **asynchronous** – secondary reads may be slightly stale
- **Replication ≠ backup** (a delete replicates too)
- Use ***w:"majority"*** to avoid rollbacks

15. Sharding – Horizontal Scaling

Sharding splits a large collection across **several servers (shards)** so data and traffic exceed one machine. Each shard is usually a **replica set**.



Data split into CHUNKS by shard key → spread over shards → queries routed by mongos

Component	Role
Shard	Holds a subset of the data (replica set)
mongos	**Query router** – apps connect to it; sends queries to the right shard(s)
Config servers	Store metadata: which chunk lives on which shard (replica set)
Shard key	Field(s) used to split data into **chunks** (≈ 128 MB); balancer moves chunks to even out shards

```
sh.enableSharding("shop")
sh.shardCollection("shop.orders", { customerId: "hashed" })      // hashed shard key → even distribution
sh.shardCollection("shop.logs", { region: 1, createdAt: 1 })     // ranged compound key → good for range queries
sh.status()          db.orders.getShardDistribution()
```

	Ranged shard key	Hashed shard key
Distribution	Contiguous ranges – can create **hotspots**	Random, **even** spread
Range queries	Efficient (targeted)	Scatter-gather (all shards)
Good for	Location / tenant based data	Monotonic keys (ObjectId, timestamp)

Good shard key

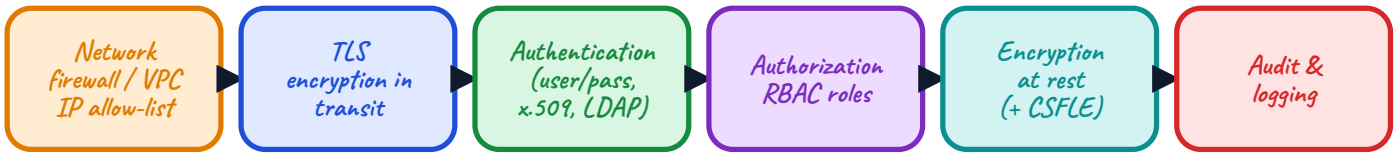
- **High cardinality** (many values)
- **Even frequency** (no hot value)
- **Not monotonic** (or use hashed)
- Appears in **most queries** → targeted

Before you shard

- Try **indexes, schema, bigger server, replica reads** first
- Shard key is hard to change (resharding exists in 5.0+)
- Queries without the key hit **all shards**

Other scaling tools: Zones (pin data ranges to regions), **Atlas auto-scaling**, read replicas, caching with Redis, **archiving** old data (Atlas Online Archive), time-series collections.

16. Security



1. Authentication & role-based access control (RBAC)

Create an admin user, then start mongod with --auth

```
use admin
db.createUser({ user: "admin", pwd: passwordPrompt(), roles: [ { role: "root", db: "admin" } ] })
// restart: mongod --auth   (or security: authorization: enabled in mongod.conf)   connect: mongosh -u admin -p --authenticationDatabase admin
use shop
db.createUser({ user: "appUser", pwd: "S3cret!pass", roles: [ { role: "readWrite", db: "shop" } ] })   // least privilege for the app
db.createUser({ user: "report", pwd: "x", roles: [ { role: "read", db: "shop" } ] })
db.getUsers();   db.updateUser("appUser", { pwd: "new" });   db.dropUser("report");
db.grantRolesToUser("appUser", ["dbAdmin"])
```

Built-in role	Permissions
read / readWrite	Read data / read + write data in a database
dbAdmin / userAdmin	Indexes, stats, collections / manage users & roles
clusterAdmin / backup / restore	Replica set & sharding admin · backup / restore tasks
root	Superuser – use rarely

2. Network & encryption

Measure	How
Don't expose the port	Bind to private IP (**net.bindIp: 127.0.0.1,10.0.0.5**); firewall / VPC; Atlas **IP access list** & private endpoints
TLS / SSL	**--tlsMode requireTLS --tlsCertificateKeyFile server.pem**; Atlas always uses TLS
Encryption at rest	WiredTiger encrypted storage (Enterprise) / disk encryption / Atlas default
Field-level encryption	**CSFLE** and **Queryable Encryption** – encrypt sensitive fields (SSN, card) client-side
Auditing	Audit log of auth & DDL / DML events (Enterprise / Atlas)

3. Application-level safety

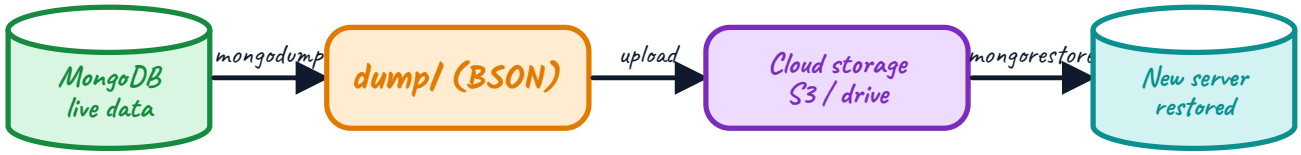
NoSQL injection – never trust input

```
// ✘ attacker sends JSON { "email": {"$ne": null}, "password": {"$ne": null} }  → logs in without a password!
const user = await db.collection("users").findOne({ email: req.body.email, password: req.body.password });
// ✔ force strings, validate, and hash passwords
const email = String(req.body.email);   // or validate with zod / Joi / express-mongo-sanitize
```

Checklist

- **Enable auth** always · strong unique passwords · least-privilege roles per app
- Keep MongoDB updated · no **0.0.0.0/0** in production · store secrets in env vars / vault
- Never use **$where** with user input · validate & sanitise · hash passwords (bcrypt) · log & monitor

17. Backup, Restore, Import & Export



Also: Atlas continuous backup (point-in-time) • filesystem snapshots • replica set delayed member

mongodump / mongorestore – binary backup (BSON)

```
mongodump --uri="mongodb://localhost:27017" --db=shop --out=./backup/2026-10-20          # one database
mongodump --uri="mongodb+srv://user:pass@cluster0.x.mongodb.net/shop" --collection=orders --gzip --archive=orders.gz
mongorestore --uri="mongodb://localhost:27017" --db=shop ./backup/2026-10-20/shop          # restore
mongorestore --drop --gzip --archive=orders.gz                                  # --drop = delete existing collections first
mongodump --oplog --out=./full  &&  mongorestore --oplogReplay ./full                  # consistent replica-set snapshot (point in time)
```

mongoexport / mongoimport – human-readable (JSON / CSV)

```
mongoexport --db=shop --collection=products --out=products.json                  # one JSON doc per line
mongoexport --db=shop --collection=products --type=csv --fields=name,price,stock --out=products.csv
mongoimport --db=shop --collection=products --file=products.json                 # (--jsonArray for [ ... ] files)
mongoimport --db=shop --collection=users --type=csv --headerline --file=users.csv
```

Method	Notes
mongodump / restore	Keeps BSON types & indexes; best for backups & migrations of small-medium data
mongoexport / import	JSON/CSV for sharing & analytics; **loses some types** (use Extended JSON) – not a backup tool
File-system snapshot	LVM / EBS snapshot of the data dir (journal on same volume) – fast for big databases
Atlas Cloud Backup	Automated snapshots + **continuous point-in-time restore** – the easy production choice
Ops Manager / Cloud Manager	Enterprise backup & monitoring for self-hosted clusters

Automate with cron (daily 2 AM) + keep 7 days

```
0 2 * * * mongodump --uri="$MONGO_URI" --gzip --archive=/backups/shop-$(date +\%F).gz && find /backups -mtime +7 -delete
```

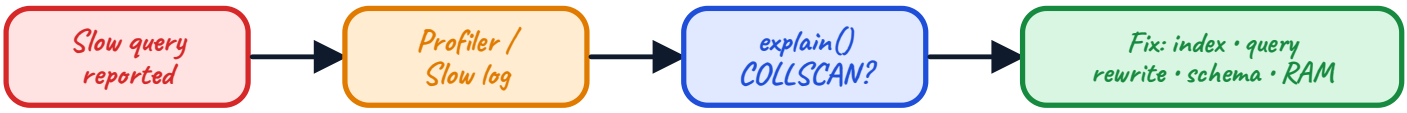
Replica set ≠ backup

A mistaken **deleteMany** replicates instantly.
Keep **separate, versioned, off-site** backups and a **delayed member**.

3-2-1 rule & testing

3 copies, **2** different media, **1** off-site. **Test restores** regularly – a backup you can't restore is useless. Secure the backup files (they contain all data).

18. Performance Tuning & Monitoring



Measure → find the slow query → understand the plan → fix → measure again

Find slow queries – the database profiler

```
db.setProfilingLevel(1, { slowms: 100 })          // 0 off · 1 log ops slower than 100 ms · 2 log everything (dev only)
db.system.profile.find().sort({ ts: -1 }).limit(5)  // ns, op, millis, planSummary: "COLLSCAN"?, docsExamined, keysExamined
db.currentOp({ secs_running: { $gt: 5 } })          // running operations          db.killOp(opid)
db.serverStatus().connections                       // current / available connections      db.stats()  ·  db.products.stats()
```

Problem	Fix
COLLSCAN in explain()	Create an index matching filter + sort (ESR rule)
docsExamined ≫ nReturned	Index not selective / wrong order – refine compound index
Large documents	Use projection; split data; avoid unbounded arrays
Deep skip()	Range pagination with ***_id*** / last seen value
Unbounded $lookup / big $group	Filter first, index foreign field, ***allowDiskUse***, pre-aggregate (computed pattern)
Too many connections	Reuse one **MongoClient** (connection pool), set ***maxPoolSize***
Working set > RAM	Add RAM, shard, remove unused indexes, TTL / archive old data

- **WiredTiger cache** ≈ 50 % of (RAM – 1 GB). Keep **indexes + hot data in RAM**.
- Use **covered queries** & projections; update with operators, not full replace.
- **Bulk writes** (***insertMany***, ***bulkWrite***) instead of many single ops.
- Set correct **write/read concerns**; read from secondaries for reporting.
- Prefer **SSD** storage, XFS filesystem on Linux, disable transparent huge pages.
- Use **time-series collections** for metrics/IoT data.

Monitoring tools

Tool	What it shows
mongostat / mongotop	Live ops/sec, queues, memory · time spent per collection
db.serverStatus()	Detailed metrics: connections, opcounters, cache, locks
Atlas Performance Advisor	Slow queries + **suggested indexes**, real-time panel, alerts
Compass → Performance / Explain	Visual query plans & server stats
Prometheus + Grafana / Datadog	Dashboards & alerts (mongodb_exporter)

19. MongoDB Atlas, Compass & the Tool Ecosystem

MongoDB ATLAS – fully managed cloud (AWS · Azure · Google Cloud)

Clusters M0 free → M10+ | Atlas Search (full-text, Lucene) | Vector Search (AI / embeddings) | Charts & Dashboards | Triggers & App Services | Backups, Alerts, Auto-scaling

MongoDB Atlas

- Atlas = MongoDB as a service: you get a ready cluster (replica set, backups, TLS, monitoring) in minutes. **M0 Free** (512 MB), shared tiers, **Dedicated** clusters, **Serverless**.
- **Network Access** (IP list / private link) · **Database Access** (users & roles) · **Connect** → drivers / Compass / mongosh · **Browse Collections** · **Data Explorer**, **Performance Advisor**, **Alerts**, **Online Archive**.

Atlas feature	What it does
Atlas Search	Full-text search (Lucene): fuzzy, autocomplete, scoring, facets – **$search** aggregation stage
Atlas Vector Search	Store **embeddings** and find similar items (**$vectorSearch**) – RAG / AI apps / recommendations
Atlas Charts	Build charts / dashboards directly on your data, embed in apps
Triggers / Functions / App Services	Run code on DB changes or schedule; HTTPS endpoints; device sync
Data Federation / SQL interface	Query S3 + Atlas together; BI tools via SQL

Atlas Search & Vector Search examples

```
db.movies.aggregate([{ $search: { index: "default", text: { query: "space adventure", path: "plot", fuzzy: {} } } }, { $limit: 5 }, { $project: { title: 1, score: { $meta: "searchScore" } } }])
db.docs.aggregate([{ $vectorSearch: { index: "vec_idx", path: "embedding", queryVector: [0.12, -0.4, /* ... */], numCandidates: 100, limit: 5 } }])
```

MongoDB Compass (GUI)

- **Connect** with the URI, browse databases / collections.
- **Documents** tab: filter box `{"{ price: {$gt: 100} }"}`, Project / Sort / Limit, edit or insert docs.
- **Aggregations** tab: build pipelines stage by stage with live previews → export to Node / Python / Java.
- **Schema** tab: field types & value distribution (data analysis).
- **Explain Plan** and **Indexes**: create / drop, see usage.
- **Validation** tab, import / export JSON & CSV, built-in **_MONGOSH** console.

Other tools	Use
mongosh	CLI shell · scripts: **mongosh --file seed.js · --eval "db.stats()" · load("script.js")**
VS Code extension / Playgrounds	Run .mongodb files, browse data inside the editor
Database Tools	mongodump, mongorestore, mongoexport, mongoimport, mongostat, mongotop
Mongoose · Prisma · Motor · PyMongo · Spring Data	ODMs / drivers for Node, Python (sync/async), Java …
Studio 3T / NoSQLBooster	Third-party GUIs with SQL → Mongo translation

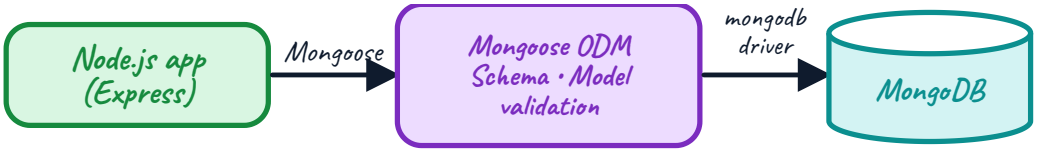
20. Using MongoDB from Node.js – Driver & Mongoose

Official driver

npm i mongodb dotenv

```
import { MongoClient, ObjectId } from "mongodb";
const client = new MongoClient(process.env.MONGO_URI, { maxPoolSize: 20 });     // create ONE client and reuse it (connection pool)
await client.connect();
const db = client.db("shop");   const users = db.collection("users");

const { insertedId } = await users.insertOne({ name: "Asad", age: 21 });
const user = await users.findOne({ _id: new ObjectId(insertedId) });
const adults = await users.find({ age: { $gte: 18 } }).sort({ age: -1 }).limit(10).project({ name: 1 }).toArray();
await users.updateOne({ name: "Asad" }, { $inc: { age: 1 } });
await users.deleteMany({ age: { $lt: 5 } });
const stats = await users.aggregate([{ $group: { _id: "$city", n: { $sum: 1 } } }]).toArray();
await client.close();                    // on app shutdown
```



Schema → Model → Document instance → .save() / .find()

	Native driver	Mongoose
Level	Low-level, flexible, lean	ODM with schemas & models
Validation	Manual / DB validator	Built into schema (required, min, enum, custom)
Extras	Fastest, newest MongoDB features first	Hooks, virtuals, populate, plugins, TypeScript types
Best for	Scripts, performance-critical code	Typical Express / Nest apps

Connection tips & graceful shutdown

```
mongoose.connection.on("connected", () => console.log("DB connected"));   mongoose.connection.on("error", console.error);
process.on("SIGINT", async () => { await mongoose.connection.close(); process.exit(0); });
// URI options: ?retryWrites=true&w=majority&maxPoolSize=20&serverSelectionTimeoutMS=5000      · one connection per process, created at startup
```

20. Mongoose – schemas, models & queries

Schemas, models, validation

npm i mongoose

```
import mongoose from "mongoose";
await mongoose.connect(process.env.MONGO_URI);

const userSchema = new mongoose.Schema({
  name:  { type: String, required: [true, "Name required"], trim: true, minlength: 2 },
  email: { type: String, required: true, unique: true, lowercase: true },
  age:   { type: Number, min: 0, max: 120, default: 18 },
  role:  { type: String, enum: ["user", "admin"], default: "user" },
  password: { type: String, select: false },                    // hidden from queries by default
  address: { city: String, zip: Number },                       // nested
  posts: [{ type: mongoose.Schema.Types.ObjectId, ref: "Post" }],   // references
}, { timestamps: true });                                       // createdAt / updatedAt
userSchema.index({ email: 1 });
userSchema.virtual("isAdult").get(function () { return this.age >= 18; });      // computed, not stored
userSchema.pre("save", async function (next) { if (this.isModified("password")) this.password = await
bcrypt.hash(this.password, 10); next(); });   // hook
userSchema.methods.greet = function () { return "Hi " + this.name; };          // instance method
userSchema.statics.findAdults = function () { return this.find({ age: { $gte: 18 } }); };   // model method
const User = mongoose.model("User", userSchema);                                // → collection "users"
```

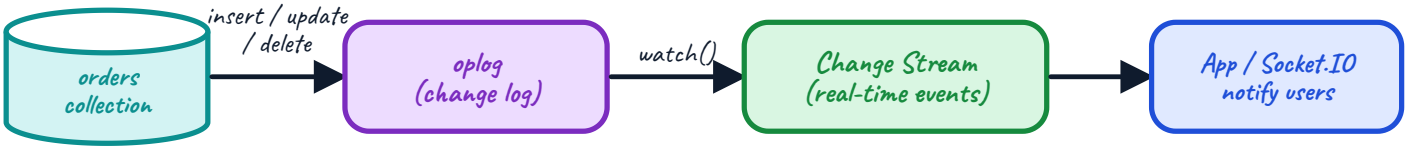
Mongoose method	MongoDB equivalent / note
`User.create(data)` / `new User(data).save()`	insertOne · runs validation & hooks
`User.find(filter).select("name").sort("-age").limit(5).lean()`	find · `lean()` = plain objects (faster)
`User.findById(id)` · `findOne()` · `countDocuments()`	by _id · first match · count
`User.findByIdAndUpdate(id, upd, {"{new:true, runValidators:true}"})`	update and return the new document
`User.findByIdAndDelete(id)` · `deleteMany()`	delete
`.populate("posts", "title")`	"JOIN" – replace ids with the referenced documents
`User.aggregate([...])`	aggregation pipeline · `session.withTransaction()` for transactions

Populate & error handling

```
const user = await User.findById(id).populate({ path: "posts", select: "title createdAt", options: { limit: 5 } });
try { await User.create({ name: "A" }); }
catch (e) { if (e.name === "ValidationError") /* e.errors.name.message */; if (e.code === 11000) /* duplicate
key */; if (e.name === "CastError") /* bad ObjectId */; }
```

21. Advanced Features

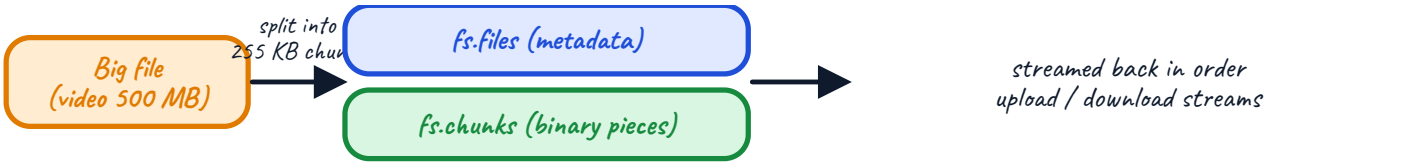
Change Streams – real-time events



needs a replica set / Atlas

```
const changeStream = db.collection("orders").watch([{ $match: { operationType: "insert", "fullDocument.total": { $gt: 100 } } }]);
for await (const change of changeStream) {                    // insert | update | delete | replace
  console.log("New big order:", change.fullDocument);         // → push notification via Socket.IO, update cache, audit log
}
```

GridFS – store large files (> 16 MB)




```
import { GridFSBucket } from "mongodb";   const bucket = new GridFSBucket(db, { bucketName: "videos" });
fs.createReadStream("movie.mp4").pipe(bucket.openUploadStream("movie.mp4", { metadata: { owner: "asad" } }));      // upload
bucket.openDownloadStreamByName("movie.mp4").pipe(res);
// download / stream
```

Time-series, capped & TTL collections

```
db.createCollection("sensor", { timeseries: { timeField: "ts", metaField: "deviceId", granularity: "minutes" } })
// optimised for metrics / IoT
db.createCollection("logs", { capped: true, size: 1048576, max: 5000 })
// fixed size ring buffer (oldest overwritten)
db.sessions.createIndex({ lastSeen: 1 }, { expireAfterSeconds: 1800 })
// TTL: auto-expire
```

Geospatial queries

```
db.shops.createIndex({ location: "2dsphere" })      // location: { type: "Point", coordinates: [lng, lat] }  ← longitude FIRST
db.shops.find({ location: { $near: { $geometry: { type: "Point", coordinates: [74.35, 31.52] }, $maxDistance: 2000 } } })      // within 2 km, nearest first
db.shops.find({ location: { $geoWithin: { $centerSphere: [[74.35, 31.52], 5 / 6378.1] } } })
// within 5 km circle
```

Feature	Use
Text search	**$text** with a text index; for advanced search use Atlas Search
Views	**db.createView("activeUsers", "users", [{"{$match:{active:true}}"}])** – read-only virtual collection
Collation	Language-aware, case-insensitive compare / sort: **{"{ locale: 'en', strength: 2 }"}**
Client-side field encryption	Encrypt fields before they leave the app (CSFLE / Queryable Encryption)
Wildcard index	**createIndex({"{ \"attrs.$**\": 1 }"})** for unknown field names
Stored JS / $where	Avoid – slow and risky

22. Practice Project – Online Shop Database

Build the ***shop*** database step by step in **mongosh**. Try to answer each question yourself before looking at the query!

1. Seed data

```
use shop
db.customers.insertMany([ { _id: 1, name: "Asad", city: "Lahore", vip: true }, { _id: 2, name: "Ali", city: "Karachi", vip: false }, { _id: 3, name: "Sara", city: "Lahore", vip: false } ])
db.orders.insertMany([
  { _id: 101, customerId: 1, items: [ { sku: "laptop", qty: 1, price: 900 }, { sku: "mouse", qty: 2, price: 20 } ], status: "paid",    createdAt: ISODate("2026-09-02") },
  { _id: 102, customerId: 2, items: [ { sku: "phone",  qty: 1, price: 600 } ], status: "shipped", createdAt: ISODate("2026-09-15") },
  { _id: 103, customerId: 1, items: [ { sku: "desk",   qty: 1, price: 150 } ], status: "paid",    createdAt: ISODate("2026-10-01") },
  { _id: 104, customerId: 3, items: [ { sku: "pen",    qty: 10, price: 1 }, { sku: "notebook", qty: 5, price: 3 } ], status: "pending", createdAt: ISODate("2026-10-10") } ])
db.orders.createIndex({ customerId: 1, createdAt: -1 })
```

2. Questions & answers

```
db.customers.find({ city: "Lahore", vip: true })                                                    // Q1 VIP customers in Lahore
db.orders.find({ status: { $in: ["paid", "shipped"] } }, { _id: 1, status: 1 }).sort({ createdAt: -1 })   // Q2 paid or shipped orders, newest first
db.orders.find({ "items.sku": "laptop" })                                                           // Q3 orders containing a laptop
db.orders.updateOne({ _id: 104 }, { $set: { status: "paid" } })                                     // Q4 mark order 104 as paid
db.orders.updateMany({ status: "pending" }, { $push: { history: { at: new Date(), note: "reminder sent" } } })
// Q5 add a history entry
db.orders.aggregate([ { $unwind: "$items" },                                                        // Q6 revenue per customer
  { $group: { _id: "$customerId", revenue: { $sum: { $multiply: ["$items.qty", "$items.price"] } }, orders: { $addToSet: "$_id" } } },
  { $sort: { revenue: -1 } } ])
db.orders.aggregate([ { $match: { status: "paid" } }, { $lookup: { from: "customers", localField: "customerId", foreignField: "_id", as: "c" } },
  { $unwind: "$c" }, { $group: { _id: "$c.city", paidOrders: { $sum: 1 } } } ])                     // Q7 paid orders per city (join)
db.orders.aggregate([ { $group: { _id: { $dateToString: { format: "%Y-%m", date: "$createdAt" } }, orders: { $sum: 1 } } }, { $sort: { _id: 1 } } ])   // Q8 orders per month
db.orders.aggregate([ { $unwind: "$items" }, { $group: { _id: "$items.sku", sold: { $sum: "$items.qty" } } }, { $sort: { sold: -1 } }, { $limit: 3 } ])   // Q9 top 3 products
db.orders.find({ createdAt: { $gte: ISODate("2026-10-01") } }).explain("executionStats").executionStats.totalDocsExamined        // Q10 index check
```

Challenges

1. Add **schema validation** to ***orders*** (status must be one of pending / paid / shipped / cancelled). 2. Add a **unique index** on customer email and **TTL** for a ***carts*** collection. 3. Write a **transaction** that creates an order and decrements ***products.stock***. 4. Calculate **average order value** per customer and show only customers above the overall average. 5. Back up with ***mongodump***, drop the DB and restore it. 6. Connect from **Node.js + Mongoose** and build a small REST API.

23. MongoDB Interview Questions & Answers

1. **What is MongoDB?** An open-source, document-oriented NoSQL database storing flexible JSON-like (BSON) documents, with indexing, aggregation, replication and sharding.
2. **What is NoSQL? Types?** Non-relational databases: document, key-value, column-family, graph, search/time-series.
3. **SQL vs MongoDB?** Tables/rows/fixed schema/JOINs vs collections/documents/flexible schema/embedding + $lookup; MongoDB scales horizontally by sharding.
4. **What is BSON?** Binary JSON – MongoDB's storage format with extra types (Date, ObjectId, Decimal128, BinData); fast to parse.
5. **What is ObjectId?** 12-byte default ***_id***: 4 B timestamp + 5 B random + 3 B counter; unique and roughly time-ordered.
6. **Max document size? 16 MB**; larger files use GridFS.
7. **What is a collection / document / field?** Group of documents (≈ table) / one record (≈ row) / key-value pair (≈ column).
8. **Is MongoDB schema-less?** Schema-flexible: no required schema, but you can enforce one with ***$jsonSchema*** validation or Mongoose.
9. **insertOne vs insertMany; ordered option?** One vs many; ordered:true stops at first error, false continues.
10. **find vs findOne?** find returns a cursor of all matches; findOne returns the first document or null.
11. **What is projection?** Choosing fields to return: ***find({}, {"{name:1, _id:0}"})***.
12. **Update operators?** ***$set $unset $inc $mul $rename $min $max $push $pull $addToSet $pop***; upsert inserts when nothing matches.
13. **updateOne vs replaceOne?** updateOne changes fields with operators; replaceOne swaps the whole document (keeps _id).
14. **$in vs $or? $and vs comma?** $in tests one field against a list; $or combines different conditions. Comma-separated conditions are an implicit AND.
15. **$elemMatch?** Matches array elements where **one element** satisfies all given conditions.
16. **What are indexes? Types?** B-tree structures speeding queries: single, compound, multikey, text, geospatial, hashed, TTL, unique, partial, wildcard.
17. **How to check if a query uses an index?** ***explain("executionStats")*** – IXSCAN good, COLLSCAN bad; compare docsExamined vs nReturned.
18. **Compound index order / ESR rule?** Equality fields first, then Sort, then Range; leftmost-prefix rule.
19. **What is a covered query?** All fields in filter and projection are in the index – no document fetch.
20. **Disadvantages of indexes?** Use RAM/disk and slow writes; too many indexes hurt.

23. MongoDB Interview Q&A (continued)

1. **What is the aggregation framework?** A pipeline of stages ($match, $group, $sort, $project, $lookup, $unwind …) for transforming and analysing data.
2. **How to do a JOIN?** `$lookup` (left outer join), or embed related data to avoid joins.
3. **Embedding vs referencing?** Embed bounded data read together (atomic, fast); reference large, unbounded or shared data.
4. **Data modeling patterns?** Subset, extended reference, computed, bucket, schema versioning, polymorphic, outlier.
5. **What is a replica set?** Group of mongod instances with the same data: one primary (writes) and secondaries; automatic election for failover.
6. **How does election work?** Heartbeats detect a dead primary; secondaries vote; a candidate with a majority becomes primary (use an odd number of voters).
7. **What is sharding? Shard key?** Horizontal partitioning across shards by a shard key; mongos routes queries; config servers hold metadata.
8. **Good shard key?** High cardinality, even distribution, non-monotonic (or hashed), used in most queries.
9. **Replication vs sharding?** Replication = copies for availability; sharding = splitting data for scale. Usually combined.
10. **Does MongoDB support transactions / ACID?** Yes – single-document always atomic; multi-document ACID since 4.0 (replica sets) and 4.2 (sharded).
11. **Write concern / read concern / read preference?** Acknowledgement level (w:1, majority) / isolation of reads (local, majority, snapshot) / which member serves reads.
12. **What is the oplog?** A capped collection on the primary recording all write operations; secondaries replay it (also used by change streams).
13. **What are change streams?** Real-time notifications of insert/update/delete via `watch()`; need a replica set.
14. **What is GridFS?** Spec for storing files > 16 MB in chunks (fs.files + fs.chunks).
15. **What is WiredTiger?** Default storage engine: document-level concurrency, compression, checkpoints + journaling.
16. **How to secure MongoDB?** Enable auth + RBAC, least privilege, TLS, firewall/IP list, encryption at rest, no public exposure, sanitize input, audit.
17. **What is NoSQL injection?** Passing operators like `{"{$ne: null}"}` in input to bypass filters – validate types, sanitize.
18. **Backup options?** mongodump/restore, filesystem snapshots, Atlas continuous backup, delayed replica; test restores.
19. **How to optimise a slow query?** Profiler → explain → add/adjust index → project fewer fields → avoid skip → rework schema → more RAM.
20. **Mongoose vs native driver?** Mongoose adds schemas, validation, middleware, populate; the driver is lower level and faster/leaner.

Interview tip

Show you design from access patterns: "I embed order items in the order (read together), reference customers, index `{"{customerId:1, createdAt:-1}"}`, use a replica set with `w:majority`, and shard by hashed customerId when data grows."

24. MongoDB Cheat Sheet

Shell basics

```
show dbs · use shop · db · show collections
db.createCollection("x") · db.x.drop()
db.dropDatabase() · db.stats() · exit
```

Create

```
db.c.insertOne({ a: 1 })
db.c.insertMany([{ a: 1 }, { a: 2 }])
```

Read

```
db.c.find(filter, projection)
db.c.findOne(filter)
  .sort({ a: -1 }).skip(10).limit(5)
db.c.countDocuments(filter) · distinct("a")
{ a: { $gt: 5, $lte: 9 } } · { $or: [..] }
{ a: { $in: [1,2] } } · { "x.y": 1 } · { arr: v }
{ a: /^abc/i } · { a: { $exists: true } }
```

Update & delete

```
db.c.updateOne(f, { $set: {a:1}, $inc: {n:1} })
db.c.updateMany(f, { $unset: {a:""} })
{ $push: {arr:v} } · $addToSet · $pull · $pop
db.c.updateOne(f, u, { upsert: true })
db.c.deleteOne(f) · db.c.deleteMany(f)
db.c.findOneAndUpdate(f, u,
{returnDocument:"after"})
```

Indexes

```
db.c.createIndex({ a: 1 }, { unique: true })
db.c.createIndex({ a: 1, b: -1 })     // compound
{ name: "text" } · { loc: "2dsphere" } · TTL
db.c.getIndexes() · db.c.dropIndex("a_1")
db.c.find(f).explain("executionStats")
```

Aggregation

```
db.c.aggregate([
  { $match: {...} },
  { $group: { _id: "$k", n: {$sum:1}, avg: {$avg:"$v"}
} },
  { $sort: { n: -1 } }, { $limit: 5 },
  { $project: { _id:0, k:"$_id", n:1 } },
  { $unwind: "$arr" },
  { $lookup: { from, localField, foreignField, as } }
])
```

Admin

```
mongodump --db shop --out dump/
mongorestore dump/
mongoexport / mongoimport  (json, csv)
mongosh "mongodb+srv://..."
```

Concepts in one line

Replica set = copies (HA) · **Shard** = split (scale) · **Index** = speed · **$lookup** = join · **Transaction** = ACID · **Change stream** = live events · **TTL** = auto-expire · **GridFS** = big files

Golden rules

Model for your queries · embed what's read together · index what you filter & sort · limit & project · never trust input · enable auth & TLS · back up & test restores · one MongoClient per app.

Store it as you use it – that is the MongoDB way! — asadokz18